PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 07 -

AUDITORIA, VERSIONAMENTO E SEGURANÇA EM IAC

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 10
O QUE VOCÊ VIU NESTA AULA? 11
REFERÊNCIAS 12

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Até agora, exploramos como declarar infraestrutura com Terraform e Terragrunt, criar módulos reutilizáveis e até integrar com Kubernetes. Mas existe uma questão fundamental que acompanha qualquer prática de **Infraestrutura como Código**: como garantir que tudo isso seja **seguro, auditável e rastreável** ao longo do tempo? Afinal, não basta criar infraestrutura rapidamente — é preciso ter confiança de que cada mudança pode ser entendida, revisada e, se necessário, revertida.

Nesta aula, vamos discutir como aplicar **versionamento, auditoria e segurança** no ciclo de IaC. Você vai ver como o Git permite registrar cada alteração de infraestrutura com clareza, mostrando **quem mudou, quando mudou e o que foi alterado**. Também vamos conhecer ferramentas que auditam e validam configurações automaticamente, evitando que vulnerabilidades e más práticas cheguem ao ambiente de produção. Além disso, falaremos sobre **Policy as Code**, scanners de segurança como **tfsec** e **Checkov**, e práticas para proteger segredos e atender requisitos de compliance.

Vamos fortalecer sua prática de IaC, não só como uma forma de automação, mas como um processo **seguro, governado e confiável**, alinhado às melhores práticas do mercado.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Nesta parte prática, vamos juntos explorar ferramentas que ajudam a garantir que o código de infraestrutura siga padrões de **segurança, governança e qualidade** antes mesmo de ser aplicado em um ambiente real. O objetivo é mostrar como a auditoria e o versionamento não são conceitos abstratos, mas práticas que podem ser incorporadas ao dia a dia de desenvolvimento de IaC.

Nos vídeos, veremos como usar ferramentas como **tfsec** e **Checkov** para analisar código Terraform e identificar problemas comuns, como recursos sem criptografia habilitada, portas expostas ao público ou senhas armazenadas de forma insegura. Essas ferramentas fazem uma varredura local, simulando o que muitos times já integram em seus pipelines de CI/CD, e servem como uma primeira barreira de segurança antes que uma mudança seja aplicada.

Além disso, o exercício prático vai mostrar como registrar essas mudanças em **Git**, aproveitando commits e pull requests como forma de auditoria. Cada alteração será rastreável e será possível verificar como a integração com ferramentas externas pode reforçar a governança. Ao final, entenderemos não apenas como escrever código Terraform, mas também como validá-lo e garantir que ele atenda boas práticas de segurança e compliance.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

Quando falamos de **Infraestrutura como Código (IaC)**, pensar apenas em “automatizar” não é suficiente. A verdadeira maturidade vem quando o time consegue garantir que essa automação é **rastreável, auditável e segura**. Isso significa que cada linha de código, cada mudança de recurso e cada aplicação no ambiente deve deixar um rastro claro que permita entender quem fez, quando fez e por que fez.

Essa visão é essencial não só para evitar erros técnicos, mas também para atender normas de governança e compliance, como **ISO 27001** e a **LGPD**, que exigem que empresas controlem e monitorem como dados e sistemas são provisionados e gerenciados.

Nesta seção, vamos percorrer os principais pilares dessa prática: **versionamento, auditoria e segurança**, explorando ferramentas e estratégias que ajudam a transformar o IaC em um processo confiável e pronto para ser usado em escala corporativa.

Versionamento e rastreabilidade em IaC

O primeiro passo é garantir que todo o código de infraestrutura esteja versionado em Git. Isso significa que cada alteração passa a ter um histórico claro: quem fez, o que mudou, quando e por que. Essa rastreabilidade não só ajuda a corrigir problemas mais rápido (rollback é muito mais simples quando você sabe qual commit introduziu um erro), mas também serve como **evidência de auditoria** em processos de conformidade.

Um ponto importante é que versionamento em Git não é apenas “guardar” o código. Ele também cria um processo de **colaboração organizada**: desenvolvedores abrem Pull Requests, revisores analisam as mudanças e o histórico mostra a **** dos recursos de forma transparente. Isso dá previsibilidade e confiança, principalmente em ambientes críticos como produção.

Ao estudar estratégias de branches como **Git Flow**, **Trunk-Based Development** ou **GitHub Flow**, você percebe como diferentes times organizam seus ciclos de entrega. O Git Flow, por exemplo, separa ambientes em branches distintas (develop, release, master), o que é útil para times que precisam de estágios bem

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

definidos. Já o Trunk-Based Development favorece times que entregam rápido, mantendo uma branch principal e aplicando feature flags.

Entender essas abordagens ajuda a criar processos de versionamento consistentes para o IaC. Quando aplicado corretamente, um fluxo Git pode evitar que mudanças inseguras cheguem à produção e permitir que ambientes de teste e staging sejam atualizados em paralelo, sempre com rastreabilidade e governança.

Auditoria e governança

Auditar significa **registrar e analisar as execuções de IaC**. Não basta versionar código; é preciso saber como ele foi aplicado no ambiente.

Ferramentas como **Terraform Cloud e Atlantis** ajudam nesse ponto. O Terraform Cloud mantém logs detalhados de cada execução (**plan e apply**), com histórico acessível para todo o time. O Atlantis integra diretamente ao GitHub ou GitLab, rodando ***terraform plan*** automaticamente quando alguém abre um Pull Request. Assim, revisores conseguem ver as mudanças planejadas antes de aprovar.

Outra prática é adotar **Policy as Code**, ou seja, escrever regras de governança que o código precisa cumprir. Com **OPA (Open Policy Agent)** ou **Sentinel**, é possível definir políticas como: “Nenhum bucket S3 pode ser criado sem criptografia habilitada” ou “Somente usuários administradores podem aprovar mudanças em produção”.

O **Open Policy Agent (OPA)** é uma ferramenta que permite transformar regras de governança em código declarativo. Em vez de depender da memória dos revisores, as políticas são aplicadas de forma automática, garantindo consistência e eliminando subjetividade. Isso significa que um ***Pull Request*** que tenta criar um bucket sem criptografia será barrado de imediato.

Estudando o OPA, você aprende a escrever políticas no formato Rego, uma linguagem simples, mas poderosa, e a integrá-las com pipelines de CI/CD. Esse aprendizado mostra como times podem evoluir de revisões manuais para auditoria automatizada, aumentando a velocidade de entrega sem abrir mão da segurança e da conformidade regulatória.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Segurança aplicada a IaC

Por fim, chegamos ao ponto mais crítico: **garantir que a infraestrutura declarada em código é segura**. Isso envolve tanto a proteção de segredos quanto a prevenção de vulnerabilidades.

Exemplo 1 — Terraform inseguro para scanners

Esse código propositalmente inseguro ajuda a entender como scanners como ***tfsec*** e ***Checkov*** identificam riscos:

```
provider "aws" {
  region = "us-east-1"
}

# Bucket S3 sem criptografia
resource "aws_s3_bucket" "insecure_bucket" {
  bucket = "iac-insecure-bucket-example"
}

# Security Group expondo porta 22 ao mundo
resource "aws_security_group" "insecure_sg" {
  name        = "insecure-sg"
  description = "SG inseguro"
  vpc_id      = "vpc-123456"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Rodando ***tfsec ./ ou checkov -d ./,*** você receberá alertas sobre bucket sem criptografia e SG exposto.

O **tfsec** e o **Checkov** são mais do que simples scanners. Eles trazem catálogos de boas práticas que evoluem com base em vulnerabilidades reais observadas no

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

mercado. Ao usá-los, você não precisa reinventar a roda: recebe relatórios prontos que destacam falhas de configuração, além de recomendações de correção.

Explorar esses scanners também mostra como integrá-los a pipelines de CI/CD. Isso permite que cada Pull Request seja automaticamente analisado, impedindo que código inseguro seja aplicado em produção. Dessa forma, o time não depende apenas da revisão manual, mas conta com uma barreira de segurança automática.

Exemplo 2 — GitHub Actions rodando Checkov

Um workflow que garante a análise de segurança em Pull Requests:

```
name: IaC Security Scan
on:
  pull_request:
    branches: [ "main" ]

jobs:
  checkov:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Run Checkov
        uses: bridgecrewio/checkov-action@v12
        with:
          directory: ./
```

Segurança em pipelines

Adicionar scanners de IaC em pipelines não é só uma prática de segurança: é também uma forma de educar o time. Cada falha apontada é uma oportunidade de aprendizado, mostrando quais são as más práticas e como corrigi-las. Isso cria um ciclo de feedback contínuo e ajuda novos membros a se alinharem rapidamente.

Estudar pipelines seguros também abre portas para entender como empresas maiores implementam auditoria contínua. Muitas vezes, não se trata apenas de rodar scanners, mas também de integrar logs, gerar relatórios e até bloquear merges automaticamente, criando uma cultura de qualidade integrada ao fluxo de trabalho.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Exemplo 3 — Policy as Code com OPA/Rego

```
package terraform.s3

deny[msg] {
  input.resource_type == "aws_s3_bucket"
  not input.encryption
  msg := sprintf("Bucket %s não possui criptografia habilitada",
[input.name])
}
```

Compliance e LGPD

A conformidade com normas e legislações, como a **LGPD** no Brasil ou a **GDPR** na Europa, exige que empresas documentem como os dados são processados e protegidos. O IaC pode ajudar nisso, pois cada recurso é declarado de forma rastreável e pode ser auditado. Isso significa que é possível provar, por exemplo, que todos os bancos de dados têm criptografia em repouso habilitada.

Ao estudar compliance aplicado a IaC, você entende como automatizar relatórios e evidências. Isso facilita auditorias externas e reduz custos de conformidade. Times que adotam essa prática ganham não só em segurança, mas também em reputação e competitividade.

Conclusão

A prática de IaC não termina quando você aplica um terraform apply. O verdadeiro valor está em garantir que esse processo seja auditável, seguro e governado. Versionamento via Git, auditoria de execuções, uso de políticas como código e scanners de segurança são peças que, juntas, constroem um ambiente confiável.

Mais importante: essas práticas não são exclusivas de grandes empresas. Qualquer time pode começar hoje, adotando Git de forma disciplinada, adicionando tfsec em pipelines e integrando auditoria gradual. O ganho em maturidade é imediato e prepara o time para desafios de maior escala.

time para desafios de maior escala.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

Nos últimos anos, a **segurança e a governança de infraestrutura como código** deixaram de ser um diferencial técnico e se tornaram **requisitos estratégicos** em empresas que operam em nuvem. O crescimento da complexidade dos ambientes, aliado à velocidade das entregas, forçou organizações a buscar maneiras mais controladas e auditáveis de gerenciar sua infraestrutura. Ferramentas de **auditoria contínua**, **Policy as Code** e **scanners automáticos** passaram a integrar o ciclo de DevOps, criando o que o mercado chama de **DevSecOps** — a junção entre desenvolvimento, segurança e operações.

Um exemplo recente vem do **GitHub**, com o artigo *"Building organization-wide governance and re-use for CI/CD and automation with GitHub Actions"*, que detalha como a empresa estruturou sua governança interna de pipelines. A equipe do GitHub adotou uma abordagem que combina **reutilização de workflows**, **revisões automatizadas** e **controle centralizado de políticas**, garantindo que todas as execuções de CI/CD obedeçam a padrões de segurança e conformidade. O resultado foi uma redução significativa no risco operacional e um aumento na velocidade de entrega dos times. Esse case reforça o papel do versionamento e da auditoria como pilares da infraestrutura moderna.

Outras empresas seguem a mesma direção. Grandes provedores de nuvem, como **AWS e Google Cloud**, vêm incorporando nativamente recursos de **compliance automatizado** e **análise de políticas**, integrando scanners como o **AWS Config**, o **Security Hub** e o **Google Cloud Policy Intelligence**. Isso mostra que a tendência não é apenas adotar ferramentas de terceiros, mas construir **ecossistemas de governança** completos, em que cada mudança na infraestrutura é validada automaticamente.

Para quem trabalha com IaC, isso significa que o futuro será cada vez mais orientado à **automação de segurança e auditoria**, reduzindo o risco humano e promovendo confiança entre as equipes. O domínio dessas práticas é um diferencial profissional e uma exigência crescente em posições de **DevOps Engineer, Cloud Architect e Security Specialist**.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você descobriu que **Infraestrutura como Código** vai muito além de automatizar recursos na nuvem. O verdadeiro valor do IaC está em tratá-lo como um processo **rastreável, auditável e seguro**, capaz de garantir previsibilidade, governança e confiança em cada mudança aplicada ao ambiente.

Começamos entendendo a importância do **versionamento com Git**, que permite acompanhar o histórico de modificações, identificar rapidamente quem fez cada alteração e facilitar auditorias e reversões de forma estruturada. Em seguida, exploramos como práticas de **auditoria e governança** — com ferramentas como Terraform Cloud, Atlantis e OPA — ajudam times a aplicar políticas automatizadas e prevenir falhas antes que elas cheguem à produção.

Avançamos também para o tema da **segurança em IaC**, aprendendo como scanners como **tfsec** e **Checkov** detectam vulnerabilidades e configuram alertas automáticos em pipelines, além de como ferramentas de **gestão de segredos** e **Policy as Code** elevam o nível de proteção e compliance das operações.

Por fim, vimos que o papel do profissional de infraestrutura moderna é unir **automação e responsabilidade**, criando ambientes eficientes e seguros. Adotar práticas de auditoria, versionamento e segurança é o que diferencia uma operação manual e arriscada de uma **infraestrutura madura, confiável e escalável**.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

AQUA SECURITY. **tfsec – Security Scanner for Terraform Code**. 2025. Disponível em: https://aquasecurity.github.io/tfsec/. Acesso em: 21 out. 2025.

ATLANTIS. **Atlantis – Terraform Pull Request Automation**. 2024. Disponível em: https://www.runatlantis.io/. Acesso em: 21 out. 2025.

BRIDGECREW. **Checkov – Infrastructure as Code Static Analysis**. 2025. Disponível em: https://www.checkov.io/. Acesso em: 21 out. 2025.

GOOGLE CLOUD. **Policy Intelligence: Automating Cloud Governance**. Google Cloud Documentation, 2025. Disponível em: https://cloud.google.com/policy-intelligence. Acesso em: 21 out. 2025.

HASHICORP. **Sentinel – Policy as Code Framework**. HashiCorp Documentation, 2025. Disponível em: https://developer.hashicorp.com/sentinel. Acesso em: 21 out. 2025.

HASHICORP. **Terraform Cloud Documentation**. HashiCorp Developer, 2025. Disponível em: https://developer.hashicorp.com/terraform/cloud-docs. Acesso em: 21 out. 2025.

OPEN POLICY AGENT (OPA). **Policy-Based Control for Cloud Native Environments**. 2024. Disponível em: https://www.openpolicyagent.org/docs/latest/. Acesso em: 21 out. 2025.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. tfsec. checkov, atlantis.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com